

Processid

```
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Process ID: %d\n", getpid());
    printf("Parent Process ID: %d\n", getppid());

    return 0;
}
```

OUTPUT

```
arnav@LAPTOP-BN8992KC:~/OS$ ./Proceessid
Process ID: 11538
Parent Process ID: 326
arnav@LAPTOP-BN8992KC:~/OS$ |
```

Fork

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed.\n");
    }
    else if (pid == 0) {
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());
    }
    else {
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());
    }

    return 0;
}
```

Output

```
arnav@LAPTOP-BN8992KC:~/OS$ nano fork.c
arnav@LAPTOP-BN8992KC:~/OS$ nano fork1.c
arnav@LAPTOP-BN8992KC:~/OS$ gcc fork1.c -o fork1
arnav@LAPTOP-BN8992KC:~/OS$ ./fork1
Parent Process
Parent PID: 11614
Child Process
Child PID: 11615
```

Parent Waits for Child (wait())

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed.\n");
    }
    else if (pid == 0) {
        printf("Child process is running...\n");
        sleep(2);
        printf("Child process finished.\n");
    }
    else {
        printf("Parent is waiting for child...\n");
        wait(NULL);
        printf("Parent continues after child finishes.\n");
    }

    return 0;
}
```

OUTPUT

```
arnav@LAPTOP-BN8992KC:~/OS$ nano wait1.c
arnav@LAPTOP-BN8992KC:~/OS$ gcc wait1.c -o wait1
arnav@LAPTOP-BN8992KC:~/OS$ ./wait1
Parent is waiting for child...
Child process is running...
Child process finished.
Parent continues after child finishes.
arnav@LAPTOP-BN8992KC:~/OS$ |
```

Multiple Child

```

#include <stdio.h>
#include <unistd.h>

int main()
{
for(int i=1;i <= 3;i++)
{
if(fork() == 0)
{
printf("Child %d PID =%d\n", i, getpid());
return 0;
}}
return 0;
}

```

Output

```

arnav@LAPTOP-BN8992KC:~/OS$ nano multichild.c
arnav@LAPTOP-BN8992KC:~/OS$ gcc multichild.c -o multichild
arnav@LAPTOP-BN8992KC:~/OS$ ./multichild
Child 1 PID =11698
Child 2 PID =11699
Child 3 PID =11700

```

Exclp()

```

#include <stdio.h>
#include <unistd.h>

int main()
{
printf("Before exec()\n");

execlp("ls","ls","-l",NULL);

printf("This will not execute.\n");

return 0;
}

```

OUTPUT

Executing the ls command...

```
total 364
-rwxr-xr-x 1 arnav arnav 16048 Aug 12 08:01 Proceessid
-rw-r--r-- 1 arnav arnav 164 Aug 12 08:05 Processid.c
-rwxr-xr-x 1 arnav arnav 16096 Aug 6 05:31 W12
-rw-r--r-- 1 arnav arnav 135 Aug 6 05:31 W12.c
-rwxr-xr-x 1 arnav arnav 15992 Aug 6 05:35 W13
-rw-r--r-- 1 arnav arnav 98 Aug 6 05:34 W13.c
-rwxr-xr-x 1 arnav arnav 15992 Aug 6 05:39 W14
-rw-r--r-- 1 arnav arnav 90 Aug 6 05:39 W14.c
-rwxr-xr-x 1 arnav arnav 16008 Jul 30 04:07 displayprocess
-rw-r--r-- 1 arnav arnav 100 Jul 30 04:06 displayprocess.c
-rwxr-xr-x 1 arnav arnav 16040 Aug 12 08:28 exclp
-rw-r--r-- 1 arnav arnav 180 Aug 12 08:25 exclp.c
-rwxr-xr-x 1 arnav arnav 15992 Aug 6 05:45 fork
-rw-r--r-- 1 arnav arnav 116 Jul 30 04:15 fork.c
-rwxr-xr-x 1 arnav arnav 16080 Aug 12 08:06 fork1
-rw-r--r-- 1 arnav arnav 364 Aug 12 08:06 fork1.c
-rwxr-xr-x 1 arnav arnav 15952 Aug 6 05:25 hello
-rw-r--r-- 1 arnav arnav 86 Aug 6 05:26 hello.c
-rw-r--r-- 1 arnav arnav 23 Aug 6 04:14 input.txt
-rwxr-xr-x 1 arnav arnav 16040 Aug 12 08:15 multichild
-rw-r--r-- 1 arnav arnav 165 Aug 12 08:17 multichild.c
-rw-r--r-- 1 arnav arnav 23 Aug 6 04:18 output.txt
-rwxr-xr-x 1 arnav arnav 15992 Aug 6 05:43 parent
```

Parent and Child Executing Concurrently

```
#include <stdio.h>
#include <unistd.h>

int main()
{
fork();

for(int i=1;i <= 5;i++)
{
printf("PID: %d Count: %d\n", getpid(), i);
sleep(1);
}
return 0;
}
```

Output

```
arnav@LAPTOP-BN8992KC:~/OS$ nano concurrent.c
arnav@LAPTOP-BN8992KC:~/OS$ gcc concurrent.c -o concurrent
arnav@LAPTOP-BN8992KC:~/OS$ ./concurrent
PID: 11842 Count: 1
PID: 11843 Count: 1
PID: 11842 Count: 2
PID: 11843 Count: 2
PID: 11842 Count: 3
PID: 11843 Count: 3
PID: 11842 Count: 4
PID: 11843 Count: 4
PID: 11842 Count: 5
PID: 11843 Count: 5
arnav@LAPTOP-BN8992KC:~/OS$ |
```

Zombie Process

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if(pid == 0)
    {
        printf("Child Exiting\n");
        return 0;
    }
    else
    {
        sleep(15);
        printf("Parent Finished\n")
    }

    return 0;
}
```

Output

```
arnav@LAPTOP-BN8992KC:~/OS$ nano zombie.c
arnav@LAPTOP-BN8992KC:~/OS$ gcc zombie.c -o zombie
arnav@LAPTOP-BN8992KC:~/OS$ ./zombie
Child Exiting
arnav
Parent Finished
```

Orphan process

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if(pid > 0)
    {
        printf("Parent Exiting\n");
        return 0;
    }
    else
    {
        sleep(5);
        printf("Child PID : %d\n", getpid());
        printf("New Parent PID : %d\n", getppid());
    }

    return 0;
}
```

Output

```
arnav@LAPTOP-BN8992KC:~/OS$ gcc orphan.c -o orphan
arnav@LAPTOP-BN8992KC:~/OS$ ./orphan
Parent Exiting
arnav@LAPTOP-BN8992KC:~/OS$ Child PID : 648
New Parent PID : 297
```

